

To accomplish both goals, you can use **Google Apps Script** to automate your exports directly within Google Drive, and an **Model Context Protocol (MCP)** server to link that structure straight to your agent.

Part 1: Automating the Exports (Google Apps Script)

You can write a script inside Google Apps Script that automatically watches a source folder of Google Docs and exports clones into your target formats (.pdf, .md, and .jsonl). [1, 2, 3]

Step-by-Step Setup:

1. Go to Google Drive. Create a **Source Folder** for your Docs, and a **Destination Folder** for your agent context.
2. Create a new Google Sheet inside your Drive, click **Extensions > Apps Script**.
3. Clear the editor and paste the following script:

```
const SRC_FOLDER_ID = "YOUR_SOURCE_FOLDER_ID";
const DEST_FOLDER_ID = "YOUR_DESTINATION_FOLDER_ID";

function exportGoogleDocs() {
  const srcFolder = DriveApp.getFolderById(SRC_FOLDER_ID);
  const destFolder = DriveApp.getFolderById(DEST_FOLDER_ID);
  const files = srcFolder.getFilesByType(MimeType.GOOGLE_DOCS);

  while (files.hasNext()) {
    const doc = files.next();
    const docId = doc.getId();
    const baseName = doc.getName();

    // 1. Export as PDF
    const pdfBlob = doc.getAs('application/pdf');
    destFolder.createFile(pdfBlob).setName(baseName + ".pdf");

    // 2. Export as Markdown (.md)
    // Uses Google's native markdown conversion endpoint
    const url =
      `https://docs.google.com/feeds/download/documents/export/Export?exp` +
      `ortFormat=markdown&id=${docId}`;
    const response = UrlFetchApp.fetch(url, {
      headers: { Authorization: 'Bearer ' +
        ScriptApp.getOAuthToken() },
      muteHttpExceptions: true
    });
  }
}
```

```

    if (response.getResponseCode() == 200) {
        destFolder.createFile(response.getBlob()).setName(baseName +
".md");
    }

    // 3. Export as JSONL (Raw Text JSON format)
    const docText =
DocumentApp.openById(docId).getBody().getText();
    const jsonlContent = JSON.stringify({ file_name: baseName,
text: docText }) + "\n";
    destFolder.createFile(baseName + ".jsonl", jsonlContent,
MimeType.PLAIN_TEXT);
  }
}

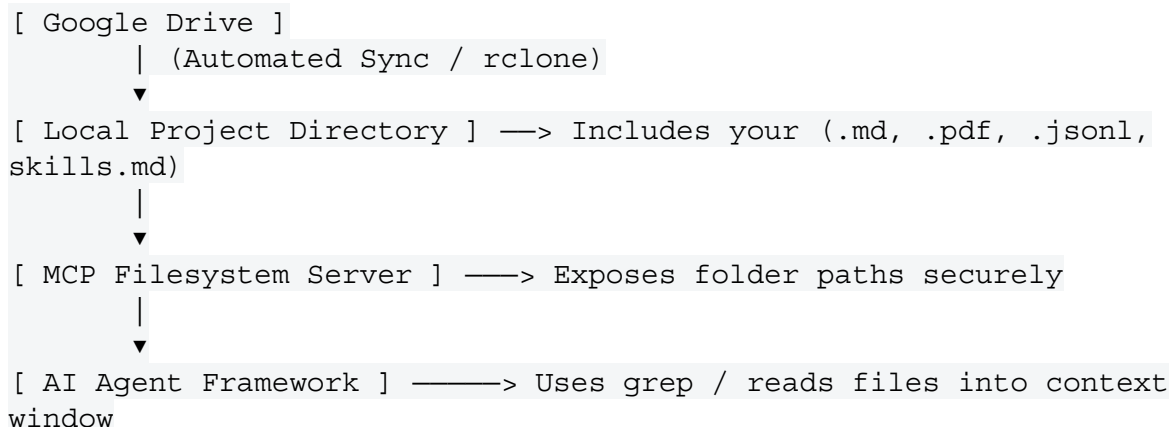
```

1. Replace "YOUR_SOURCE_FOLDER_ID" and "YOUR_DESTINATION_FOLDER_ID" with the long string of letters/numbers found in your Google Drive folder URLs.
2. Click **Save**, then hit **Run**. It will prompt you for security permissions to access your Drive data.
3. To automate this daily or hourly, click the **Triggers (Alarm Clock icon)** on the left sidebar and set exportGoogleDocs to run on a time-driven schedule.

Note: For skills.md, you should manually create a master orchestration file in your destination folder that maps your custom code schemas to your agent. [4, 5]

Part 2: Connecting the Files to your Agent Framework

To let your local or cloud-based AI agent seamlessly look at your files, run grep, or inherit configurations, you should use the industry-standard **Model Context Protocol (MCP)**. [2, 6]



Step-by-Step Integration:

1. **Locally Sync the Files:** Use [Google Drive for Desktop](#) or a tool like rclone to mirror your Google Drive agent folder to your local computer or server workspace.
2. **Expose via Filesystem MCP:** Most modern developer agent frameworks (like Claude Desktop, Cursor, or Windsurf) utilize a localized filesystem protocol to inspect code. Add the official `@modelcontextprotocol/server-filesystem` tool to your agent's system configuration file: [7]

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "/path/to/your/synced/google/drive/folder"
      ]
    }
  }
}
```

1. **Dynamic Prompting ("Grep this"):** Once the path is bound, your agent instantly inherits local filesystem permissions. You can issue commands to your prompt interface such as:
 - *"Grep our internal JSONL logs for error codes."*
 - *"Read skills.md to see what remote schemas are available for my text tool."*

Would you like help mapping out the **exact directory architecture** for your agent workspace, or do you need help **generating the skills.md schema syntax** to register a specific remote tool? [4]

[1] <https://support.google.com>

[2] <https://stackoverflow.com>

[3] <https://blog.coupler.io>

[4] <https://inkeep.com>

[5] <https://medium.com>

[6] <https://mcpmarket.com>

[7] <https://www.linkedin.com>